

Lab_Day03-Securing Data

Lab Objectives

By the end of this lab session, students will be able to:

- Embed and extract secret payloads within digital media files using multi-layered steganography tools to understand the mechanics of covert communication channels.
- Compute and compare multi-algorithm cryptographic hashes (MD5, SHA-256, SHA-512) to verify data integrity and visually witness the avalanche effect caused by minor file modifications.
- Implement file-level symmetric encryption and secure data-wiping protocols on local storage to protect highly confidential media assets from unauthorized access and forensic recovery.

Lab Tasks

TASK 1: STEGANOGRAPHY	2
DIFFERENT THINGS TO TRY:.....	2
TASK 2: INTEGRITY CHECK	2
TASK 3: DATA PROTECTION (ENCRYPTING YOUR CONFIDENTIAL DATA)	2
APPENDIX A: USING OPENPUFF STEGANOGRAPHY	3
APPENDIX B: INSTALLING GUI HASH GENERATOR AND COMPARING DIGESTS	4

Task 1: Steganography

Using any steganography tool of your choice, send secret message/data to your friend.

- Basic track: Search and use an online tool
- Recommended track: Use a tool e.g. OpenPuff:
https://embeddedsd.net/OpenPuff_Steganography_Home.html. See full working instructions in Appendix A.

Different things to try:

1. Use an image to secretly send your WIFI network details (SSID, i.e. Network Name and Password). Use WhatsApp to send over the image. The other person should be able to get the secret message.
2. Imagine that your friend is in another city/country and “whatever communication” you will have with them is tapped. Can you still find a way to send your secret information? Put this scenario in AI and explore whether it is possible or not. Write down your options and discuss with the Lab Instructors.

Task 2: Integrity Check

3. Continuing from your previous task, remember that you are adding something to your image/audio/video (whatever medium you used for your steganography). At source, check the integrity of the file.
 - a. Find hash of your original file
 - b. Find hash of your new file with secret message.
 - c. Compare both hashes.
4. At destination:
 - a. Find hash of the received file
 - b. Extract secret message and try to remove that message from the file to evade any security audit detection.

Note: There are many programs which implement hash functions. You can use command-line based programs e.g. HashCat in Kali Linux, or other GUI based programs for different platforms. A more user-friendly GUI-base hash function utility is available in Appendix B.

Task 3: Data Protection (Encrypting your confidential data)

Symmetric encryption is usually used to protect individual sensitive files (like IDs, financial records, or private photos) on a local machine without disrupting regular file folder organization. You can also use full-disk encryption (BitLocker/FileVault/VeraCrypt) which protect your data, BUT only when the computer is turned off. However, if your laptop is unlocked at a coffee shop, an attacker has access to everything. Therefore, you essentially need a file-level encryption tool for your absolute highest-tier secrets.

In this task, you will act as a security-conscious individual protecting highly confidential personal media within an existing folder structure. You will utilize **Picocrypt**, an open-source, modern cryptographic utility, to execute file-level encryption.

- Download and install or use portable version of the **Picocrypt** (Windows/macOS/Linux) from <https://github.com/picocrypt/Picocrypt>
- Test encrypting and decrypting
 - A small file
 - A folder with multiple files
 - A large size file (something like 1+ GB file to observe encryption and decryption time)

Security note: Notice that the original plaintext file still exists next to your new *.pc file! To actually protect the data, you must safely delete the original. In Picocrypt's advanced settings, you can check "**Wipe source files**" to shred the original plaintext using secure overwriting.

Other Software to try:

- Protect your data in the cloud (also fine for local data encryption) <https://cryptomator.org/>
- Full disk/volume encryption (like BitLocker): <https://veracrypt.io/en/Home.html>

Appendix A: Using OpenPuff Steganography

Time required: 25 minutes

Objective: summarize the basics of cryptographic concepts.

Description: unlike cryptography that scrambles a message so that it cannot be viewed, steganography hides the

existence of the data. In this project, you will use openPuff to create a hidden message.

1. use your web browser to go to https://embeddedsdsw.net/OpenPuff_Steganography_Home.html

2. click **Manual** to open the openPuff manual. save this file to your computer. read through the manual to see the

features available. return to the home page when finished.

3. click **download binary for windows/linux** to download the program. When a page appears asking for payments, click the **.ZIP** link to download the program for evaluation without submitting a payment.

4. click **Screenshot** to view a screen capture of openPuff. right-click this image and save it as **openPuff_Screenshot.jpg** on your computer. This will be the carrier file that will contain the secret message.

5. navigate to the location of the download and uncompress the openPuff zip file on your computer.

6. now create the secret message to be hidden. open notepad and enter **This is a secret message**.

7. save this file as **Message.txt** and close notepad.

8. create a zip file from the **Message** file. navigate to the location of this file through file explorer and right-click it.

9. click **Send to** and select **Compressed (zipped) folder** to create the zip file.

10. navigate to the openPuff directory and double-click **openPuff.exe**.

11. click **hide**.

12. under **(1)**, create three unrelated passwords and enter them into **Cryptography (A)**, **(B)**, and **(C)**. Be sure that

the **Scrambling (C)** password is long enough to turn the **Password check** bar from red to green.

13. under (2), locate the message to be hidden. click **Browse** and navigate to the file **Message.zip**. click **open**.
14. under (3), select the carrier file. click **Add** and navigate to **openPuff_Screenshot.jpg**.
15. click **hide data!**.
16. navigate to a different location from that of the carrier files and click **ok**.
17. after the processing has completed, navigate to the location of the carrier file that contains the message, and open the file. can you detect anything different with the file now that it contains the message?
18. now uncover the message. close the openPuff data hiding screen to return to the main menu.
19. click **unhide!**.
20. enter the three passwords.
21. click **Add Carriers** and navigate to the location of **Carrier1** that contains the hidden message.
22. click **unhide!** and navigate to a location to deposit the hidden message. When it has finished processing, click **ok**.
23. click **done** after reading the report.
24. go to that location and you will see **Message.zip**.
25. close openPuff and close all windows.

Appendix B : Installing GUI hash generator and comparing Digests

Time required: 20 minutes

objective: summarize the basics of cryptographic concepts.

description: In this project, you will download a gui hash generator and compare the results of various hash algorithms.

1. create a Microsoft Word document with the contents **Now is the time for all good men to come to the aid of their country.**
2. save the document as **Country1.docx** on the desktop or in a directory specified by your instructor.
3. now make a single change to **Country1.docx** by removing the period at the end of the sentence so it says **Now is the time for all good men to come to the aid of their country** and then save the document as **Country2.docx** in the same directory.
4. close the document and Microsoft Word.
5. use your web browser to go to <https://hashcalc.soft112.com/>
6. scroll down and then click **download hashcalc**.
7. click **download**.
8. click **download link**.
9. follow the default instructions to install hashcalc.
10. launch hashcalc to display the hashcalc window.
11. In addition to the hash algorithms selected by default, check the box next to the following hash algorithms to add them: **Md5, sha256, sha384, sha512, and Md2**.

12. click the file explore button next to **data:**.
13. navigate to the document **Country1.docx**.
14. click **open**.
15. In the hashcalc window, click **Calculate**.
16. review the digests generated. If necessary, expand the size of the window. What can you say about these digests? compare Md2 with sha512. What makes sha512 better than Md2? Why?
17. click the file explore button next to **data:**.
18. navigate to the document **Country2.docx**.
19. click **open**.
20. In the hashcalc window, click **Calculate**.
21. This file is the same as the previous except a single period was removed. are the digests different? What does this tell you about hashing digests?
22. close all windows.